ia-26/functions.py

```python
import numpy as np


# ========================
# Benchmark Test Functions
# ========================


def f1(x):
    """
    Sphere Function
    """
    x = np.asarray(x)

    return np.sum(x ** 2)


def f2(x):
    """
    Schaffer Function N.2

    Agora recebe vetor:
    x = [x1, x2]
    """

    x = np.asarray(x)

    x1 = x[0]
    x2 = x[1]

    numerator = (
        np.sin(np.sqrt(x1**2 + x2**2))**2
        - 0.5
    )

    denominator = (
        1 + 0.001 * (x1**2 + x2**2)
    )**2

    return 0.5 + numerator / denominator


def f3(x):
    """
    Ackley Function
    """

    x = np.asarray(x)

    n = len(x)

    term1 = (
        -20
        * np.exp(
            -0.2 * np.sqrt(np.sum(x**2) / n)
        )
    )

    term2 = (
        -np.exp(
            np.sum(
                np.cos(2 * np.pi * x)
            ) / n
        )
    )

    return term1 + term2 + 20 + np.e


def f4(x):
    """
    Rosenbrock Function
    """

    x = np.asarray(x)

    return np.sum(
        100 * (x[:-1]**2 - x[1:])**2
        + (x[:-1] - 1)**2
    )


def f5(x):
    """
    Rastrigin Function
    """

    x = np.asarray(x)

    return np.sum(
        x**2
        - 10 * np.cos(2 * np.pi * x)
        + 10
    )


def f6(x):
    """
    Griewank Function
    """

    x = np.asarray(x)

```

```python
    n = len(x)

    sum_term = np.sum(x**2) / 4000

    prod_term = np.prod(
        np.cos(
            x / np.sqrt(np.arange(1, n + 1))
        )
    )

    return sum_term - prod_term + 1


def u(x, a, k, m):

    if x > a:
        return k * (x - a) ** m

    elif x < -a:
        return k * (-x - a) ** m

    else:
        return 0


def f7(x):
    """
    Penalized Function
    """

    x = np.asarray(x)

    n = len(x)

    y = 1 + (x + 1) / 4

    first_term = (
        10 * np.sin(np.pi * y[0])**2
    )

    middle_sum = np.sum(
        (y[:-1] - 1)**2
        * (
            1
            + 10
            * np.sin(np.pi * y[1:])**2
        )
    )

    last_term = (y[-1] - 1)**2

    penalty = np.sum(
        [
            u(xi, 10, 100, 4)
            for xi in x
        ]
    )

    return (
        (np.pi / n)
        * (
            first_term
            + middle_sum
            + last_term
        )
        + penalty
    )


def f8(x):
    """
    Schwefel Function
    """

    x = np.asarray(x)

    n = len(x)

    return (
        418.9829 * n
        - np.sum(
            x * np.sin(np.sqrt(np.abs(x)))
        )
    )
```